



Tutoraggio: Fondamenti di Informatica

A.A. 2025/2026

Lezione #4



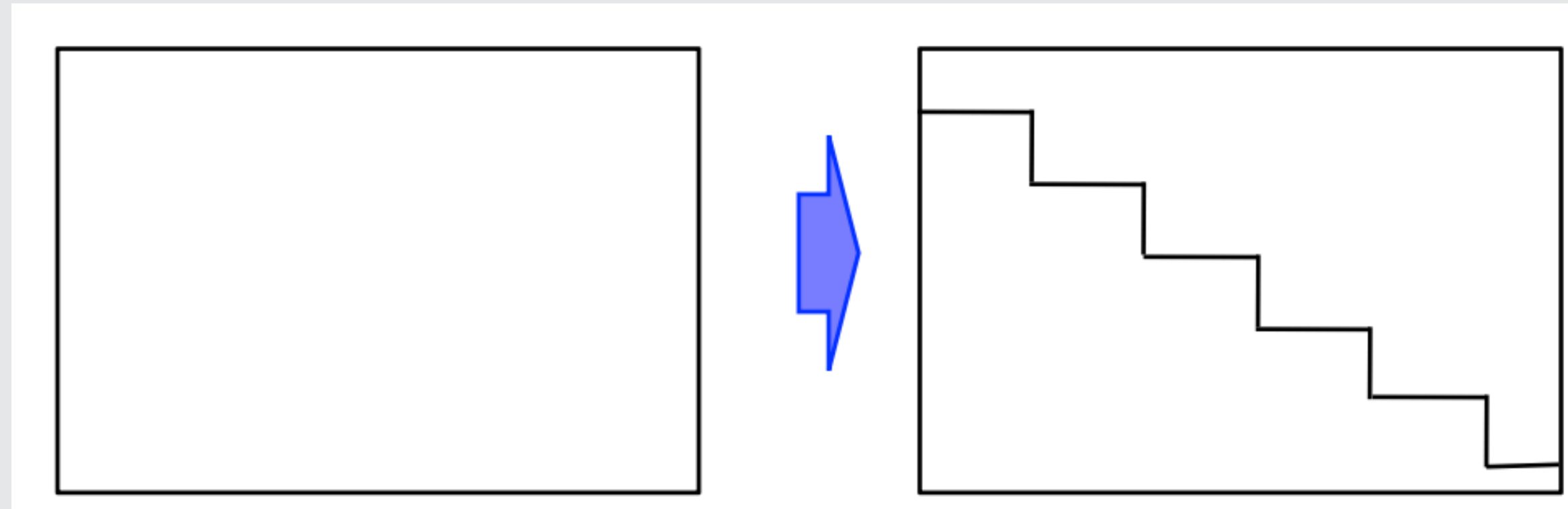
Esercizio :

Uso del comando `for` in struttura bi-dimensionale (A3)

Scrivere una funzione che traccia una "scaletta" all'interno di un'immagine bianca.

Sulle slide viene riportato anche che bisogna utilizzare i seguenti parametri:

- Punto di inizio
- Dimensione "scalini"
- Colore delle linee



**Attenzione a non uscire
dal bordo!**

Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Soluzione:

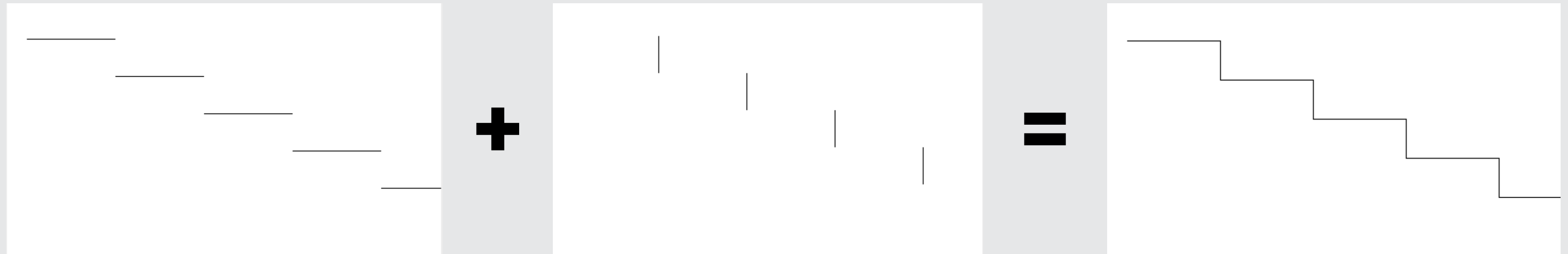
```
1 def disegnaGradino(pict,xStart,yStart,width,height,col):
2     #@Param pict:Picture
3     #@Param xStart:integer
4     #@Param yStart:integer
5     #@Param width:integer
6     #@Param height:integer
7     #@Param col:Color
8     pictWidth=getWidth(pict)
9     pictHeight=getHeight(pict)
10    xUpperBound = min(xStart+width,pictWidth)
11    print xStart+width, pictWidth, xUpperBound
12    if yStart < pictHeight:
13        for x in range (xStart,xUpperBound): #qui il meno uno e' implicito
14            p=getPixel(pict,x,yStart)
15            setColor(p,col)
16    yUpperBound = min(yStart+height,pictHeight)
17    print yStart+height, pictHeight, yUpperBound
18    if xUpperBound < pictWidth:
19        for y in range (yStart,yUpperBound):
20            p=getPixel(pict,xUpperBound,y) #qui lo devo esplicitare
21            setColor(p,col)
```

```
23 def disegnaScala(pict,xStart,yStart,width,height,col):
24     #@Param pict:Picture
25     #@Param xStart:integer
26     #@Param yStart:integer
27     #@Param width:integer
28     #@Param height:integer
29     #@Param col:Color
30     pictWidth=getWidth(pict)
31     pictHeight=getHeight(pict)
32     if xStart <0 or xStart >= pictWidth:
33         return
34     if yStart <0 or yStart >= pictHeight:
35         return
36     numGradini = min((pictWidth-xStart)/width,(pictHeight-yStart)/height)+1
37     x = xStart
38     y = yStart
39     for i in range(0,numGradini):
40         disegnaGradino(pict,x,y,width,height,col)
41         x = x + width
42         y = y + height
```

Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Soluzione Alternativa:



Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Soluzione Alternativa:

```
1 def disegnaScala(pict,xStart,yStart,width,height,col):
2     #@Param pict:Picture
3     #@Param xStart:int
4     #@Param yStart:int
5     #@Param width:int
6     #@Param height:int
7     pictWidth = getWidth(pict)
8     pictHeight = getWidth(pict)
9     xStart = max(xStart,0)
10    yStart = max(yStart,0)
11    for x in range(xStart,pictWidth):
12        y = ((x-xStart)/width)*height + yStart
13        if y > pictHeight-1:
14            break
15        p = getPixel(pict,x,y)
16        setColor(p,col)
17    for y in range(yStart,pictHeight):
18        x = ((y-yStart)/height+1)*width + xStart
19        if x > pictWidth-1:
20            break
21        p = getPixel(pict,x,y)
22        setColor(p,col)
```


Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Soluzione Alternativa:

rette orizzontali

rette verticali

```
1 def disegnaScala(pict,xStart,yStart,width,height,col):
2     #@Param pict:Picture
3     #@Param xStart:int
4     #@Param yStart:int
5     #@Param width:int
6     #@Param height:int
7     pictWidth = getWidth(pict)
8     pictHeight = getWidth(pict)
9     xStart = max(xStart,0)
10    yStart = max(yStart,0)
11    for x in range(xStart,pictWidth):
12        y = ((x-xStart)/width)*height + yStart
13        if y > pictHeight-1:
14            break
15        p = getPixel(pict,x,y)
16        setColor(p,col)
17    for y in range(yStart,pictHeight):
18        x = ((y-yStart)/height+1)*width + xStart
19        if x > pictWidth-1:
20            break
21        p = getPixel(pict,x,y)
22        setColor(p,col)
```

Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Soluzione Alternativa:

il valore di y incrementa ogni volta che x incrementa della "larghezza del gradino"

il valore di x incrementa ogni volta che y incrementa della "altezza del gradino"

```
1 def disegnaScala(pict,xStart,yStart,width,height,col):
2     #@Param pict:Picture
3     #@Param xStart:int
4     #@Param yStart:int
5     #@Param width:int
6     #@Param height:int
7     pictWidth = getWidth(pict)
8     pictHeight = getWidth(pict)
9     xStart = max(xStart,0)
10    yStart = max(yStart,0)
11    for x in range(xStart,pictWidth):
12        y = ((x-xStart)/width)*height + yStart
13        if y > pictHeight-1:
14            break
15        p = getPixel(pict,x,y)
16        setColor(p,col)
17    for y in range(yStart,pictHeight):
18        x = ((y-yStart)/height+1)*width + xStart
19        if x > pictWidth-1:
20            break
21        p = getPixel(pict,x,y)
22        setColor(p,col)
```

Esercizio :

Uso del comando for in struttura bi-dimensionale (A3)

Soluzione Alternativa:

il valore di y incrementa ogni
volta che x incrementa della
“larghezza del gradino”

$$y = ((x - xStart) / width) * height + yStart$$

il valore di x incrementa ogni

$$x = ((y - yStart) / height + 1) * width + xStart$$

```
1 def disegnaScala(pict, xStart, yStart, width, height, col):
2     #@Param pict:Picture
3     #@Param xStart:int
4     #@Param yStart:int
5     #@Param width:int
6     #@Param height:int
7
8     yStart = max(yStart, 0)
9     for x in range(xStart, pictWidth):
10        y = ((x - xStart) / width) * height + yStart
11        if y > pictHeight - 1:
12            break
13
14        x = ((y - yStart) / height + 1) * width + xStart
15        if x > pictWidth - 1:
16            break
17        p = getPixel(pict, x, y)
18        setColor(p, col)
```


Esercizio

Uso del comando for in strutture bi-dimensionale (C)

Questo esercizio è una combinazione di due esercizi già dati in precedenza, potete provare a riusare (adattandole) le soluzioni già date:

scrivere una funzione che, data un'immagine e un intero K , inverte il valore della componente Red con quello della componente Blue in tutti e soli i pixel dell'immagine che hanno la coordinata larghezza (la prima) $< K$ (in altre parole, dato un pixel con coordinate (x,y) , con $x < K$, se prima della modifica il pixel ha Red=10 e Blue=50, dopo la modifica deve avere Red=50 e Blue=10). Suggerimento per verificare la correttezza della soluzione elaborata: provate a eseguire la funzione passando come parametro un'immagine costruita così: `makeEmptyPicture(500, 500, red)` (costruisce un quadrato tutto rosso). Se la soluzione è corretta, dovrete ottenere una nuova immagine con una striscia blu di larghezza K nella parte sinistra dell'immagine (il resto rimane rosso).

Esercizio

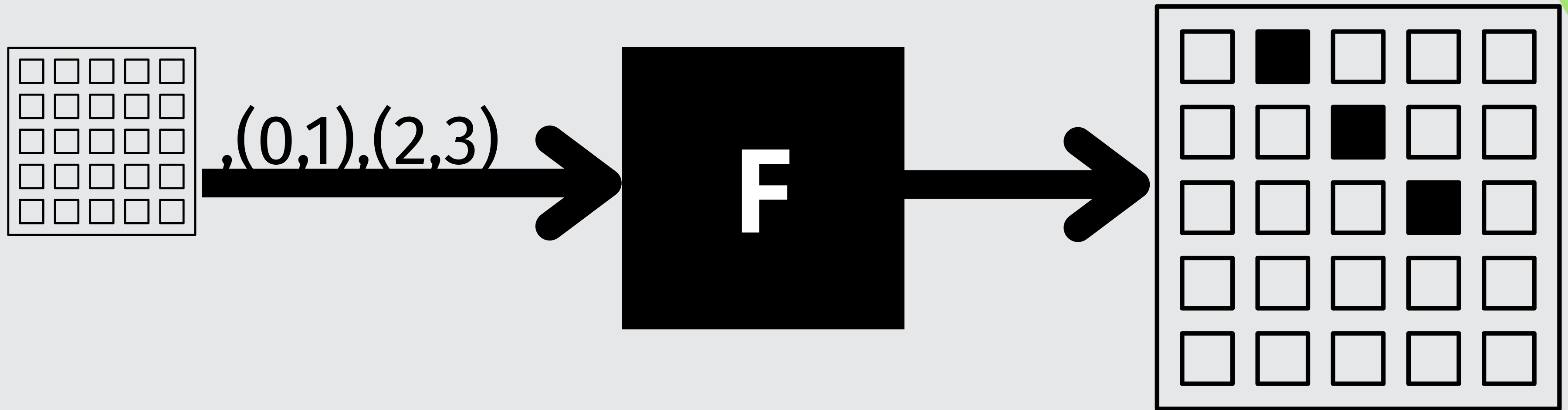
Uso del comando for in strutture bi-dimensionale (C)

```
1 def invertiRossoBlu(pict,K):
2     #@Param pict:Picture
3     #@Param K:integer
4     upperBound = min(K,getWidth(pict))
5     height = getHeight(pict)
6     for x in range(0,upperBound):
7         for y in range(0,height):
8             p = getPixel(pict,x,y)
9             b = getBlue(p)
10            r = getRed(p)
11            setRed(p,b)
12            setBlue(p,r)
```

Esercizio

Uso del comando for in strutture bi-dimensionale (C)

Data un'immagine, e due coppie di coordinate (x_0, y_0) e (x_1, y_1) , scrivere una funzione che traccia una linea retta (di un colore definito come parametro della funzione) che congiunge il punto (x_0, y_0) con il punto (x_1, y_1) .



Esercizio

Uso del comando for in strutture bi-dimensionale (C)

Data un'immagine, e due coppie di coordinate (x_0, y_0) e (x_1, y_1) , scrivere una funzione che traccia una linea retta (di un colore definito come parametro della funzione) che congiunge il punto (x_0, y_0) con il punto (x_1, y_1) .

“Per due punti passa una e una sola retta”

- $y(x) = mx + q \quad || \quad y(x) = K$
- $m = \frac{y_1 - y_0}{x_1 - x_0}$
- $q = mx_0 - y_0$

Esercizio

Uso del comando for in strutture bi-dimensionale (C)

```
12 def esercizioTracciaRetta(pict,x0,y0,x1,y1):
13     #@Param pict:Picture
14     #@Param x0:integer
15     #@Param y0:integer
16     #@Param x1:integer
17     #@Param y1:integer
18     if x0 == x1:
19         tracciaRettaVerticale(pict,y0,y1,x0)
20     return
21     step = 1
22     if x0>x1:
23         step = - step
24     my = y1-y0
25     mx = x1-x0
26     m = (y1-y0) / float(x1-x0)
27     q = (x0*y1-x1*y0)/float(x0-x1)
28     for x in range(x0,x1+1,step):
29         y = m * x + q
30         y = int(y)
31         p = getPixel(pict,x,y)
32         setColor(p,black)
```


Esercizio

Uso del comando for in strutture bi-dimensionale (C)

caso in cui la
formula non può
essere applicata!

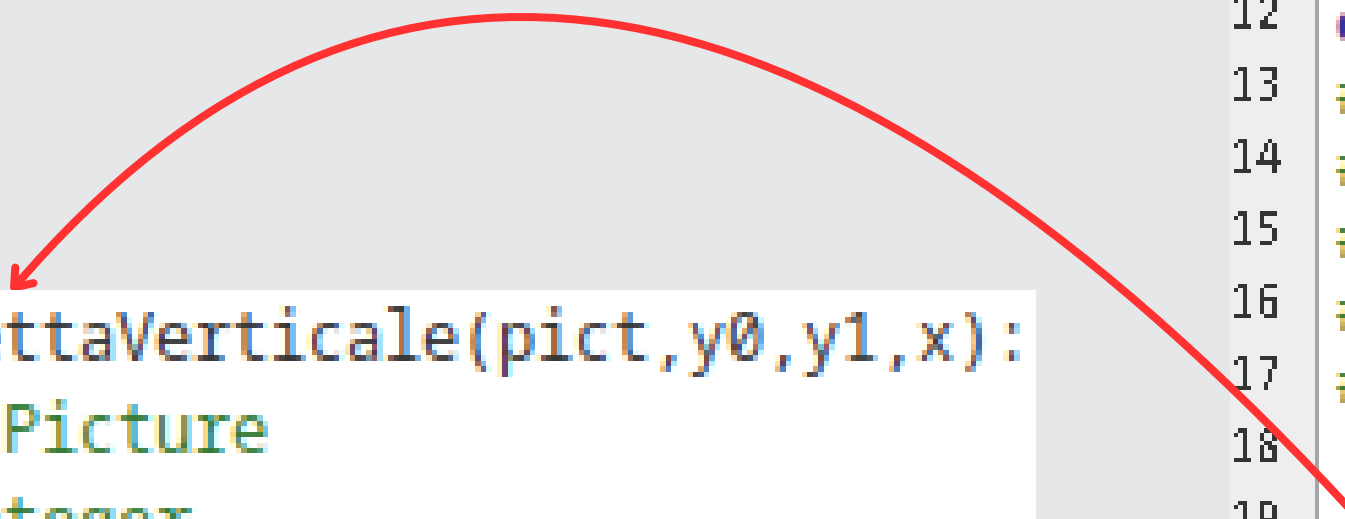


```
12 def esercizioTracciaRetta(pict,x0,y0,x1,y1):
13     #@Param pict:Picture
14     #@Param x0:integer
15     #@Param y0:integer
16     #@Param x1:integer
17     #@Param y1:integer
18     if x0 == x1:
19         tracciaRettaVerticale(pict,y0,y1,x0)
20         return
21     step = 1
22     if x0>x1:
23         step = - step
24     my = y1-y0
25     mx = x1-x0
26     m = (y1-y0) / float(x1-x0)
27     q = (x0*y1-x1*y0)/float(x0-x1)
28     for x in range(x0,x1+1,step):
29         y = m * x + q
30         y = int(y)
31         p = getPixel(pict,x,y)
32         setColor(p,black)
```

Esercizio

Uso del comando for in strutture bi-dimensionale (C)

```
1 def tracciaRettaVerticale(pict,y0,y1,x):
2     #@Param pict:Picture
3     #@Param y0:integer
4     #@Param y1:integer
5     #@Param x:integer
6     lBound = min(y0,y1)
7     uBound = max(y0,y1)
8     for y in range(lBound,uBound+1):
9         p = getPixel(pict,x,y)
10        setColor(p,black)
```



```
12 def esercizioTracciaRetta(pict,x0,y0,x1,y1):
13     #@Param pict:Picture
14     #@Param x0:integer
15     #@Param y0:integer
16     #@Param x1:integer
17     #@Param y1:integer
18     if x0 == x1:
19         tracciaRettaVerticale(pict,y0,y1,x0)
20         return
21     step = 1
22     if x0>x1:
23         step = - step
24     my = y1-y0
25     mx = x1-x0
26     m = (y1-y0) / float(x1-x0)
27     q = (x0*y1-x1*y0)/float(x0-x1)
28     for x in range(x0,x1+1,step):
29         y = m * x + q
30         y = int(y)
31         p = getPixel(pict,x,y)
32         setColor(p,black)
```

Esercizio

Uso del comando for in strutture bi-dimensionale (C)

Calcolo m e q



```
12 def esercizioTracciaRetta(pict,x0,y0,x1,y1):
13     #@Param pict:Picture
14     #@Param x0:integer
15     #@Param y0:integer
16     #@Param x1:integer
17     #@Param y1:integer
18     if x0 == x1:
19         tracciaRettaVerticale(pict,y0,y1,x0)
20     return
21     step = 1
22     if x0>x1:
23         step = - step
24     my = y1-y0
25     mx = x1-x0
26     m = (y1-y0) / float(x1-x0)
27     q = (x0*y1-x1*y0)/float(x0-x1)
28     for x in range(x0,x1+1,step):
29         y = m * x + q
30         y = int(y)
31         p = getPixel(pict,x,y)
32         setColor(p,black)
```


Esercizio

Uso del comando for in strutture bi-dimensionale (C)

- ciclo dalla x iniziale alla x finale
- calcolo la y corrispondente al valore di x attuale

```
12 def esercizioTracciaRetta(pict,x0,y0,x1,y1):
13     #@Param pict:Picture
14     #@Param x0:integer
15     #@Param y0:integer
16     #@Param x1:integer
17     #@Param y1:integer
18     if x0 == x1:
19         tracciaRettaVerticale(pict,y0,y1,x0)
20     return
21     step = 1
22     if x0>x1:
23         step = - step
24     my = y1-y0
25     mx = x1-x0
26     m = (y1-y0) / float(x1-x0)
27     q = (x0*y1-x1*y0)/float(x0-x1)
28     for x in range(x0,x1+1,step):
29         y = m * x + q
30         y = int(y)
31         p = getPixel(pict,x,y)
32         setColor(p,black)
```



Esercizio

Uso del comando for in strutture bi-dimensionale (C)

```
12 def esercizioTracciaRetta(pict,x0,y0,x1,y1):  
13     #@Param pict:Picture  
14     #@Param x0:integer  
15     #@Param y0:integer
```

$(x_0, x_1), (y_0, y_1) \in \mathbb{N}^2$

$m, q \in \mathbb{R}$

$y \in \mathbb{N}$

valore di x attuale

```
24     my = y1-y0  
25     mx = x1-x0  
26     m = (y1-y0) / float(x1-x0)  
27     q = (x0*y1-x1*y0)/float(x0-x1)  
28     for x in range(x0,x1+1,step):  
29         y = m * x + q  
30         y = int(y)  
31         p = getPixel(pict,x,y)  
32         setColor(p,black)
```

Primo esercizio:

Scrivere una funzione che, data un'immagine, la modifica invertendo il valore della componente Red con quello della componente Blue in tutti e soli i pixel dell'immagine per i quali vale la condizione $\text{Red} < \text{Blue}$.



Primo esercizio:

```
1  def invertiRossoBlu(pict):
2  #@Param pict:Picture
3  #@Param K:integer
4      width = getWidth(pict)
5      height = getHeight(pict)
6      for x in range(0,width):
7          for y in range(0,height):
8              p = getPixel(pict,x,y)
9              b = getBlue(p)
10             r = getRed(p)
11             if r < b :
12                 setRed(p,b)
13                 setBlue(p,r)
```

Ricerca luminosità minima:

Si definisca come luminosità di un pixel px la somma dei valori delle tre componenti del colore di px ($r+g+b$). Scrivere una funzione che, data un'immagine, restituisce il minimo tra le luminosità di tutti i pixel dell'immagine.

Minimo in un vettore:

3	6	2	1	5	8	9	43	-32	54	34	2
---	---	---	---	---	---	---	----	-----	----	----	---

Il valore minimo del vettore?

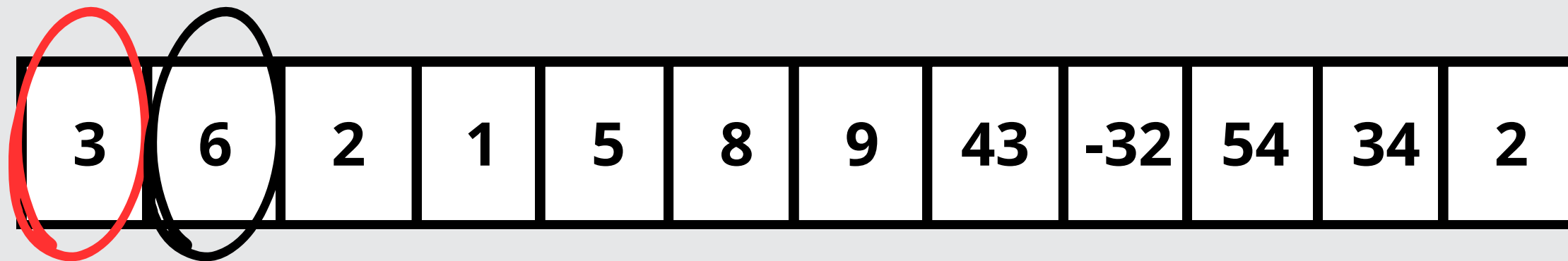
Minimo in un vettore:

3	6	2	1	5	8	9	43	-32	54	34	2
---	---	---	---	---	---	---	----	-----	----	----	---

Come abbiamo fatto a dire che -32 è il valore minimo del vettore?

1. Abbiamo percorso un elemento alla volta con lo sguardo
2. Confrontato uno ad uno fra di loro
3. Ad ogni confronto abbiamo tenuto quello che vinceva
4. Il vincitore è l'effettivo minimo del vettore!

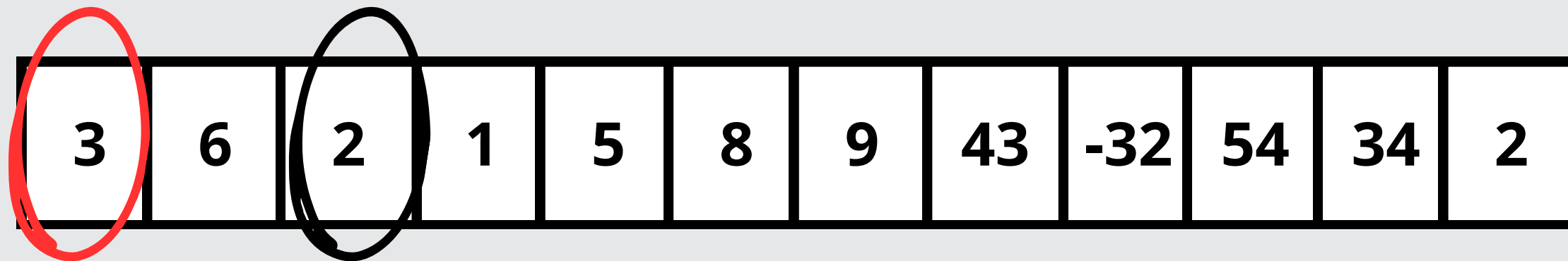
Minimo in un vettore:



Come abbiamo fatto a dire che -32 è il valore minimo del vettore?

1. Abbiamo percorso un elemento alla volta con lo sguardo
2. Confrontato uno ad uno fra di loro
3. Ad ogni confronto abbiamo tenuto quello che vinceva
4. Il vincitore è l'effettivo minimo del vettore!

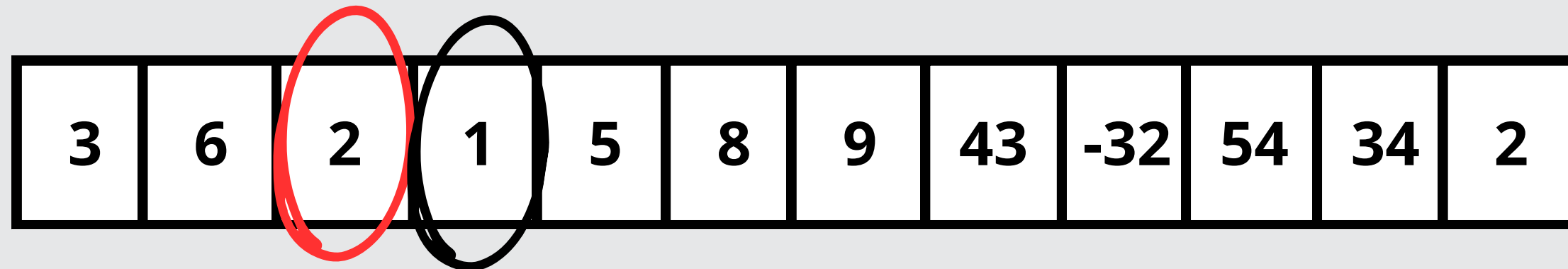
Minimo in un vettore:



Come abbiamo fatto a dire che -32 è il valore minimo del vettore?

1. Abbiamo percorso un elemento alla volta con lo sguardo
2. Confrontato uno ad uno fra di loro
3. Ad ogni confronto abbiamo tenuto quello che vinceva
4. Il vincitore è l'effettivo minimo del vettore!

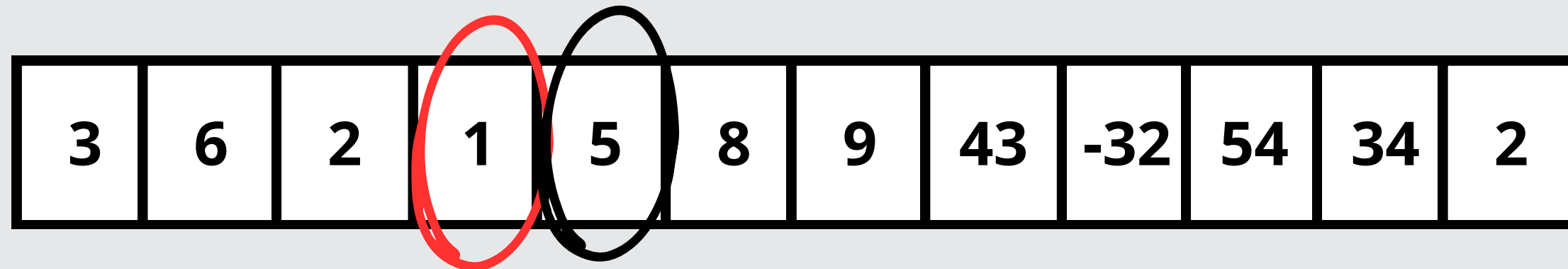
Minimo in un vettore:



Come abbiamo fatto a dire che -32 è il valore minimo del vettore?

1. Abbiamo percorso un elemento alla volta con lo sguardo
2. Confrontato uno ad uno fra di loro
3. Ad ogni confronto abbiamo tenuto quello che vinceva
4. Il vincitore è l'effettivo minimo del vettore!

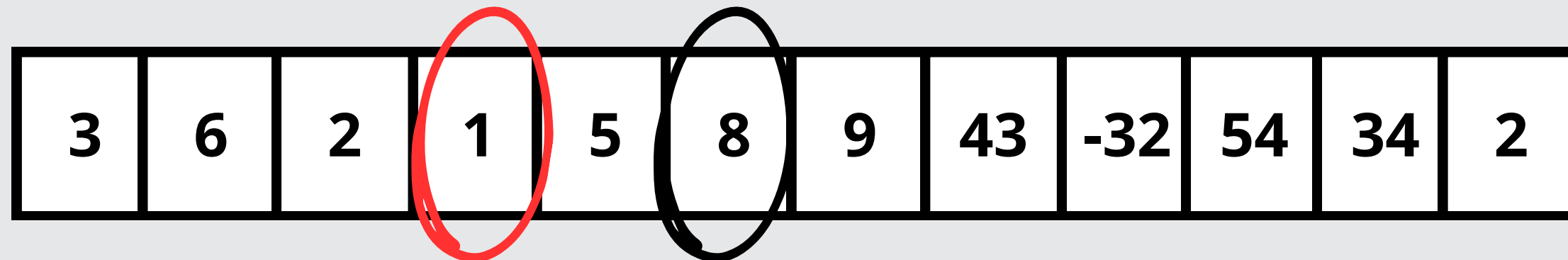
Minimo in un vettore:



Come abbiamo fatto a dire che -32 è il valore minimo del vettore?

1. Abbiamo percorso un elemento alla volta con lo sguardo
2. Confrontato uno ad uno fra di loro
3. Ad ogni confronto abbiamo tenuto quello che vinceva
4. Il vincitore è l'effettivo minimo del vettore!

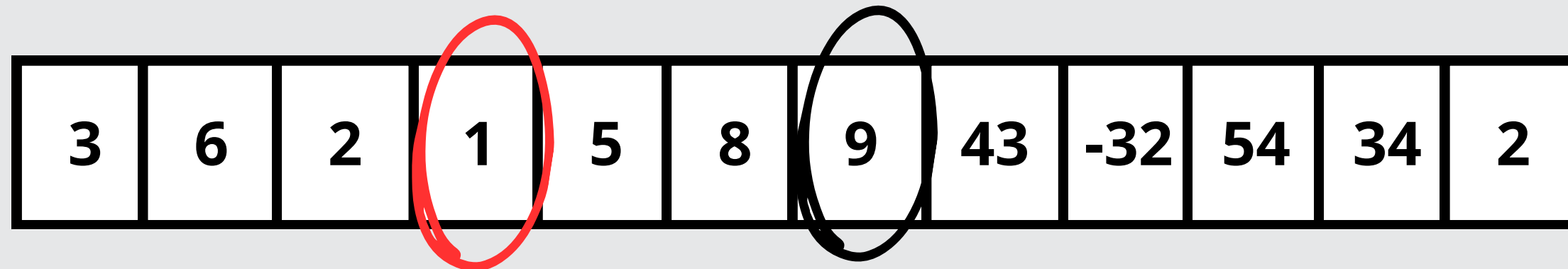
Minimo in un vettore:



Come abbiamo fatto a dire che -32 è il valore minimo del vettore?

1. Abbiamo percorso un elemento alla volta con lo sguardo
2. Confrontato uno ad uno fra di loro
3. Ad ogni confronto abbiamo tenuto quello che vinceva
4. Il vincitore è l'effettivo minimo del vettore!

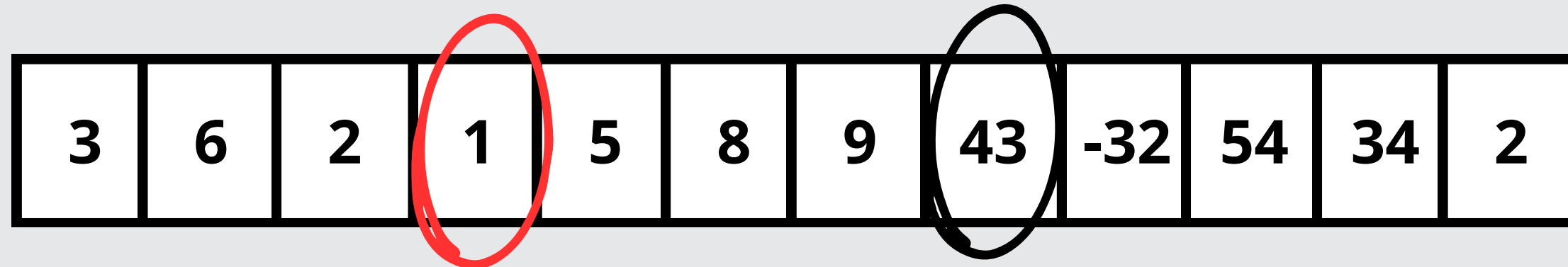
Minimo in un vettore:



Come abbiamo fatto a dire che -32 è il valore minimo del vettore?

1. Abbiamo percorso un elemento alla volta con lo sguardo
2. Confrontato uno ad uno fra di loro
3. Ad ogni confronto abbiamo tenuto quello che vinceva
4. Il vincitore è l'effettivo minimo del vettore!

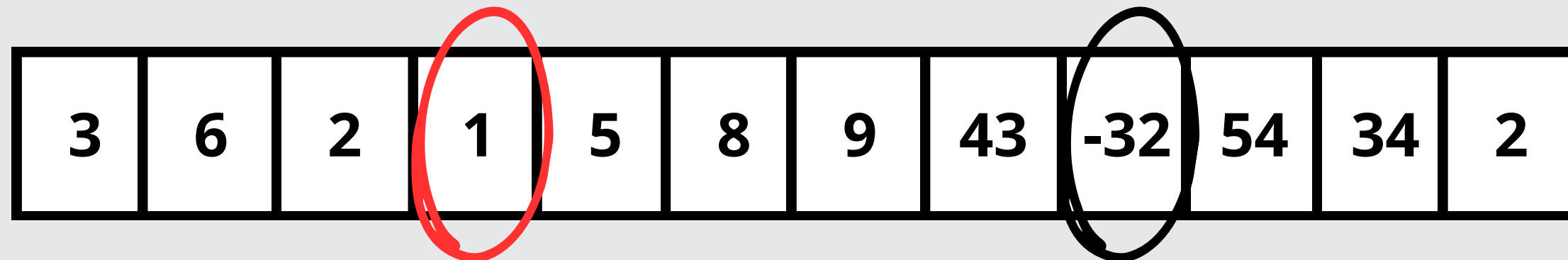
Minimo in un vettore:



Come abbiamo fatto a dire che -32 è il valore minimo del vettore?

1. Abbiamo percorso un elemento alla volta con lo sguardo
2. Confrontato uno ad uno fra di loro
3. Ad ogni confronto abbiamo tenuto quello che vinceva
4. Il vincitore è l'effettivo minimo del vettore!

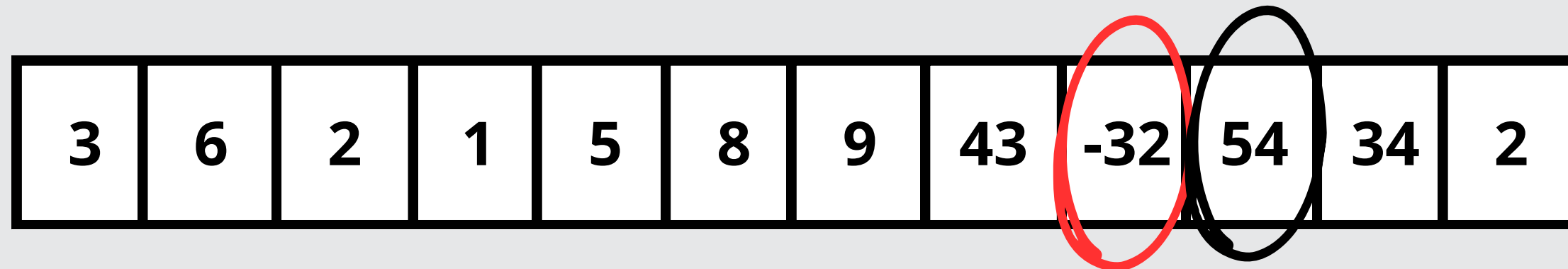
Minimo in un vettore:



Come abbiamo fatto a dire che -32 è il valore minimo del vettore?

1. Abbiamo percorso un elemento alla volta con lo sguardo
2. Confrontato uno ad uno fra di loro
3. Ad ogni confronto abbiamo tenuto quello che vinceva
4. Il vincitore è l'effettivo minimo del vettore!

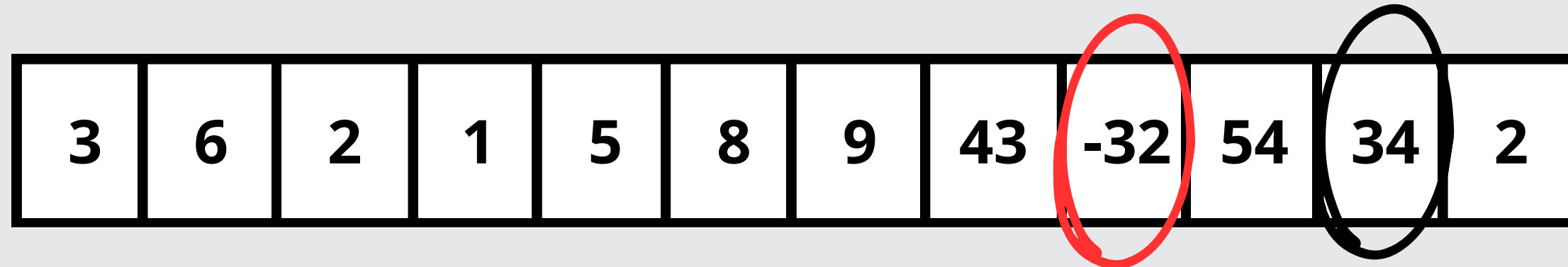
Minimo in un vettore:



Come abbiamo fatto a dire che -32 è il valore minimo del vettore?

1. Abbiamo percorso un elemento alla volta con lo sguardo
2. Confrontato uno ad uno fra di loro
3. Ad ogni confronto abbiamo tenuto quello che vinceva
4. Il vincitore è l'effettivo minimo del vettore!

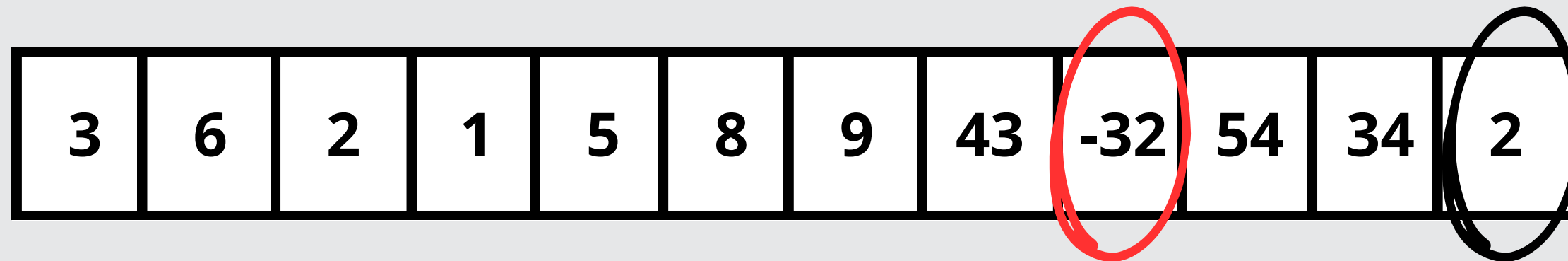
Minimo in un vettore:



Come abbiamo fatto a dire che -32 è il valore minimo del vettore?

1. Abbiamo percorso un elemento alla volta con lo sguardo
2. Confrontato uno ad uno fra di loro
3. Ad ogni confronto abbiamo tenuto quello che vinceva
4. Il vincitore è l'effettivo minimo del vettore!

Minimo in un vettore:



Come abbiamo fatto a dire che -32 è il valore minimo del vettore?

1. Abbiamo percorso un elemento alla volta con lo sguardo
2. Confrontato uno ad uno fra di loro
3. Ad ogni confronto abbiamo tenuto quello che vinceva
4. Il vincitore è l'effettivo minimo del vettore!

Minimo in un vettore:

3	6	2	1	5	8	9	43	-32	54	34	2
---	---	---	---	---	---	---	----	-----	----	----	---

Cosa ci serve?

1. **Scorrere il vettore**
2. **Una variabile per tenere il valore minimo mentre lo scorriamo**
3. **confrontare i valori**

Minimo in un vettore: soluzione

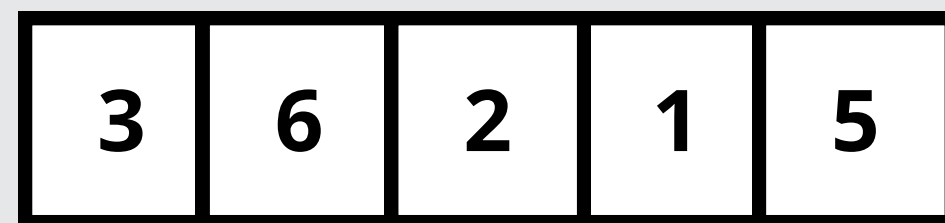
```
1 def minimoVettore(vettore):  
2     #@Param vettore:list of integer  
3     min = 1000000000000000  
4     for i in range(0,len(vettore)):  
5         if vettore[i]< min:  
6             min = vettore[i]  
7     print "il minimo del vettore è: ",min
```

Minimo in un vettore: soluzione

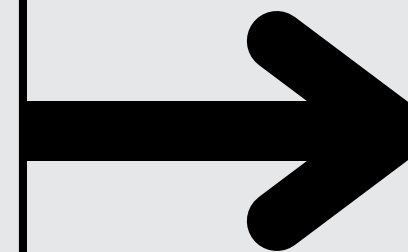
```
1 def minimoVettore(vettore):  
2     #@Param vettore: list of integer  
3     min = 1000000000000000  
4     for i in range(0, len(vettore)):  
5         if vettore[i] < min:  
6             min = vettore[i]  
7     print "il minimo del vettore è: ", min
```

Qualche problema?

Minimo in un vettore: soluzione

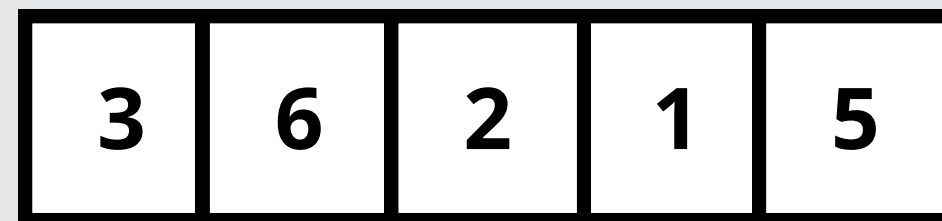


```
1 def minimoVettore(vettore):  
2     #@Param vettore:list of integer  
3     min = 10000000000000  
4     for i in range(0,len(vettore)):  
5         if vettore[i]< min:  
6             min = vettore[i]  
7     print "il minimo del vettore è: ",min
```



**Cosa restituisce la
funzione?**

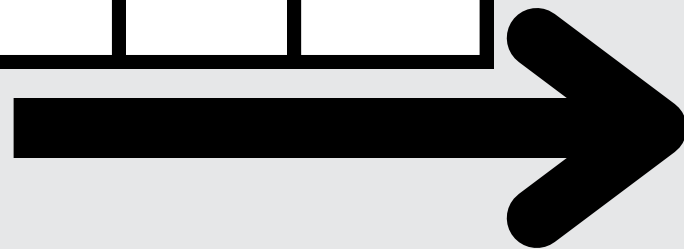
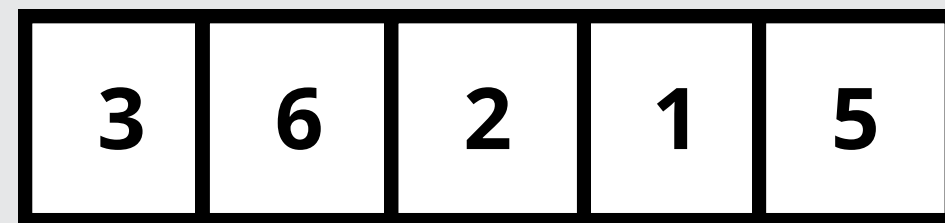
Minimo in un vettore: soluzione



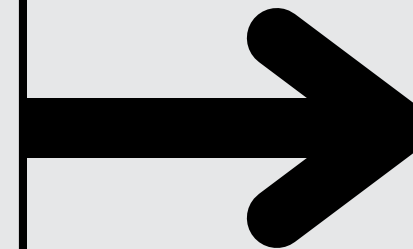
```
1 def minimoVettore(vettore):  
2     #@Param vettore:list of integer  
3     min = 10000000000000  
4     for i in range(0,len(vettore)):  
5         if vettore[i]< min:  
6             min = vettore[i]  
7     print "il minimo del vettore è: ",min
```

Non restituisce nulla!!!

Minimo in un vettore: soluzione

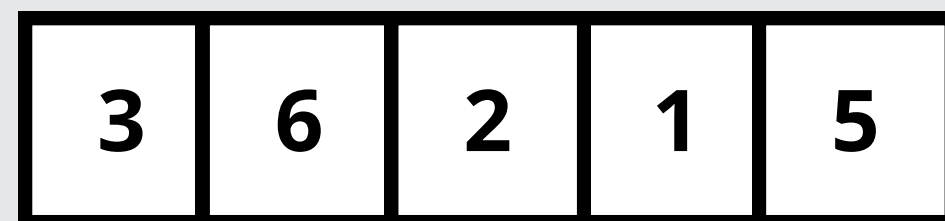


```
1 def minimoVettore(vettore):  
2   #@Param vettore:list of integer  
3   min = 10000000000000  
4   for i in range(0,len(vettore)):  
5       if vettore[i]< min:  
6           min = vettore[i]  
7   print "il minimo del vettore è: ",min
```

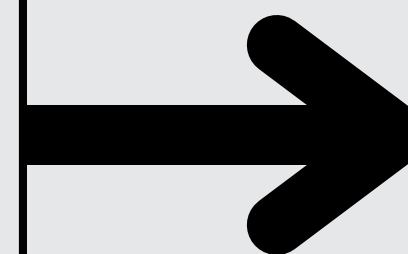


**Cosa “Stampa sul terminale”
la funzione?**

Minimo in un vettore: soluzione



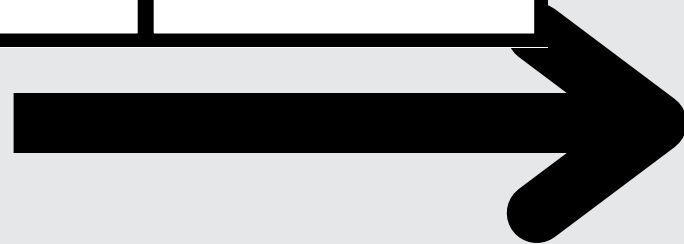
```
1 def minimoVettore(vettore):  
2     #@Param vettore:list of integer  
3     min = 10000000000000  
4     for i in range(0,len(vettore)):  
5         if vettore[i]< min:  
6             min = vettore[i]  
7     print "il minimo del vettore è: ",min
```



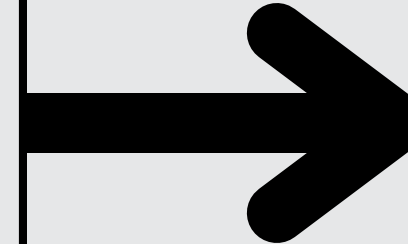
Il minimo del vettore e': 1

Minimo in un vettore: soluzione

2000000 0000010	1000000 000005	1000000 000007
--------------------	-------------------	-------------------



```
1 def minimoVettore(vettore):  
2     #@Param vettore:list of integer  
3     min = 10000000000000  
4     for i in range(0,len(vettore)):  
5         if vettore[i]< min:  
6             min = vettore[i]  
7     print "il minimo del vettore è: ",min
```



**Cosa “Stampa sul terminale”
la funzione?**

Minimo in un vettore: soluzione

2000000 0000010	1000000 000005	1000000 000007
--------------------	-------------------	-------------------

```
1 def minimoVettore(vettore):  
2     #@Param vettore:list of integer  
3     min = 10000000000000  
4     for i in range(0,len(vettore)):  
5         if vettore[i]< min:  
6             min = vettore[i]  
7     print "il minimo del vettore è: ",min
```

Il minimo del vettore e':
100000000000000

Minimo in un vettore: soluzione

2000000 0000010	1000000 000005	1000000 000007
--------------------	-------------------	-------------------

```
1 def minimoVettore(vettore):  
2     #@Param vettore:list of integer  
3     min = 10000000000000  
4     for i in range(0,len(vettore)):  
5         if vettore[i]< min:  
6             min = vettore[i]  
7     print "il minimo del vettore è: ",min
```

Il minimo del vettore e':
100000000000000

Come risolviamo?

Minimo in un vettore: soluzione

```
1 def minimoVettore(vettore):
2     #@Param vettore:list of integer
3     if len(vettore) < 1:
4         return
5     min = vettore[0]
6     for i in range(1,len(vettore)):
7         if vettore[i]< min:
8             min = vettore[i]
9     print "il minimo del vettore è: ",min
```


Ricerca luminosità minima: soluzione

```
1 def lumen(pix):
2     #@Param pix:Pixel
3     #@Return Integer
4     return getRed(pix)+getBlue(pix)+getGreen(pix)
5 def minLumPicture(pict):
6     #@Param pict:Picture
7     #@Return integer
8     listPixels = getPixels(pict)
9     if len(listPixels) < 1:
10        return
11    min = lumen(listPixels[0]) #Posso metterci un altro valore?
12    for i in range(1,len(vettore)):
13        actualLumen = lumen(listPixels[i])
14        if actualLumen < min:
15            min = actualLumen
16    return min
```

Ricerca luminosità minima: soluzione

inizializzo la variabile min

Ciclo sul vettore

Confronto tra min e valore attuale

```
1  def lumen(pix):
2  #@Param pix:Pixel
3  #@Return Integer
4      return getRed(pix)+getBlue(pix)+getGreen(pix)
5  def minLumPicture(pict):
6  #@Param pict:Picture
7  #@Return integer
8      listPixels = getPixels(pict)
9      if len(listPixels) < 1:
10         return
11     min = lumen(listPixels[0]) #Posso metterci un altro valore?
12     for i in range(1,len(listPixels)):
13         actualLumen = lumen(listPixels[i])
14         if actualLumen < min:
15             min = actualLumen
16     return min
```

Ricerca luminosità minima: soluzione alternativa **Matrice**

```
1  def lumen(pix):
2  #@Param pix:Pixel
3  #@Return Integer
4      return getRed(pix)+getBlue(pix)+getGreen(pix)
5  def minLumPicture(pict):
6  #@Param pict:Picture
7  #@Return integer
8      height = getHeight(pict)
9      width = getWidth(pict)
10     min = lumen(getPixel(pict,0,0))
11     for x in range(0,width):
12         for y in range(0,height):
13             p = getPixel(pict,x,y)
14             actualLumen = lumen(p)
15             if actualLumen < min:
16                 min = actualLumen
17     return min
```


Caccia all'errore

La funzione **contaPix**(L, k) intende calcolare il numero di pixel nella sequenza L (prelevati da un'immagine tramite la funzione `getPixels()`) che hanno esattamente k altri pixel (escluso quindi se stesso) con stessa luminosità. La funzione contiene però un errore (uno solo), corretto il quale la funzione calcola il risultato desiderato.

Qual'è l'errore?

Caccia all'errore



```
1  def contaPix(L, k):
2      #@ param L : sequence of pixel
3      #@ param k: int
4      #@ return int
5      p = 0
6      for i in range(0, len(L)):
7          count = 0
8          lum1 = lum(L[i])
9          for j in range(0, len(L)):
10             if i != j:
11                 lum2 = lum(L[j])
12                 if lum1 == lum2:
13                     count = count+1
14                 if count == k:
15                     p=p+1
16             return p
17
18
19  def lum(p):
20      #@ param p : pixel
21      #@ return int
22      return (getRed(p)+getGreen(p)+getBlue(p))
```

Caccia all'errore

Capiamo che **p** è l'accumulatore che conta i numeri di volte che abbiamo dei pixel con k duplicati di luminosità

Funzione ausiliaria per il calcolo della luminosità di un pixel

```
1  def contaPix(L, k):
2  #@ param L : sequence of pixel
3  #@ param k: int
4  #@ return int
5      p = 0
6      for i in range(0, len(L)):
7          count = 0
8          lum1 = lum(L[i])
9          for j in range(0, len(L)):
10             if i != j:
11                 lum2 = lum(L[j])
12                 if lum1 == lum2:
13                     count = count+1
14                     if count == k:
15                         p=p+1
16             return p
17
18
19 def lum(p):
20 #@ param p : pixel
21 #@ return int
22     return (getRed(p)+getGreen(p)+getBlue(p))
```

Caccia all'errore

count è l'accumulatore che per ogni pixel conta le occorrenze di luminosità

se è lo stesso pixel non controllo

Se **count** è uguale a **K** incremento il contatore **p**

```
1  def contaPix(L, k):
2  #@ param L : sequence of pixel
3  #@ param k: int
4  #@ return int
5      p = 0
6      for i in range(0, len(L)):
7          count = 0
8          lum1 = lum(L[i])
9          for j in range(0, len(L)):
10             if i != j:
11                 lum2 = lum(L[j])
12                 if lum1 == lum2:
13                     count = count+1
14                 if count == k:
15                     p=p+1
16             return p
17
18
19  def lum(p):
20  #@ param p : pixel
21  #@ return int
22      return (getRed(p)+getGreen(p)+getBlue(p))
```

Caccia all'errore

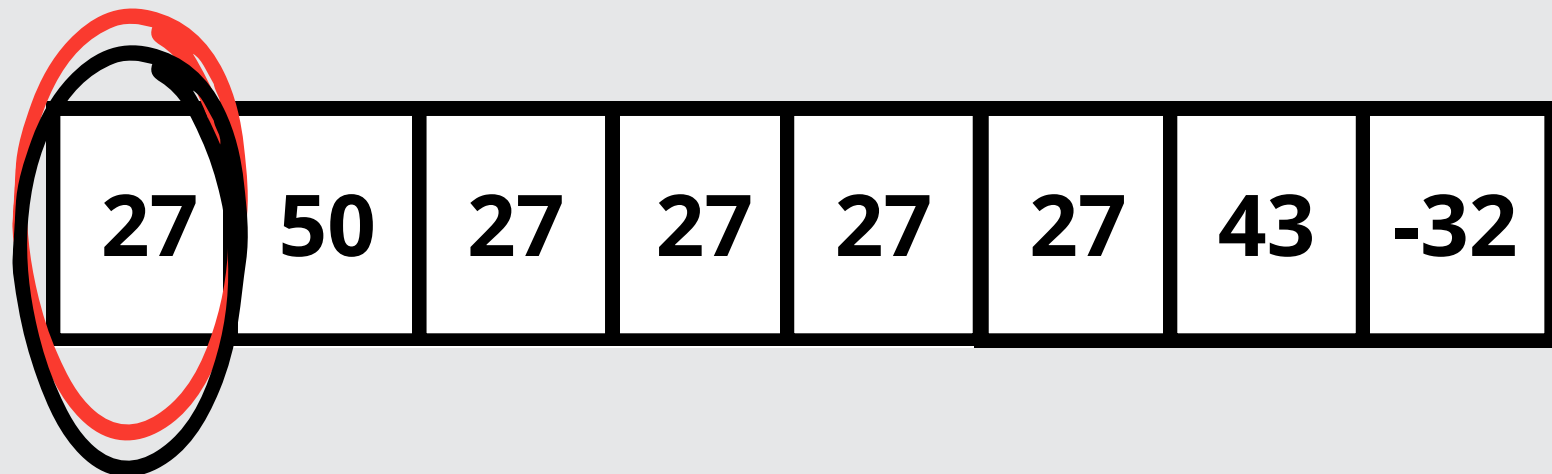
Siamo sicuri che
questo if fa ciò
che vogliamo ?

```
1  def contaPix(L, k):
2  #@ param L : sequence of pixel
3  #@ param k: int
4  #@ return int
5      p = 0
6      for i in range(0, len(L)):
7          count = 0
8          lum1 = lum(L[i])
9          for j in range(0, len(L)):
10             if i != j:
11                 lum2 = lum(L[j])
12                 if lum1 == lum2:
13                     count = count+1
14                 if count == k:
15                     p=p+1
16             return p
17
18
19  def lum(p):
20  #@ param p : pixel
21  #@ return int
22      return (getRed(p)+getGreen(p)+getBlue(p))
```


Caccia all'errore

```
1 def contaPix(L, k):
2     #@ param L : sequence of pixel
3     #@ param k: int
4     #@ return int
5     p = 0
6     for i in range(0, len(L)):
7         count = 0
8         lum1 = lum(L[i])
9         for j in range(0, len(L)):
10            if i != j:
11                lum2 = lum(L[j])
12                if lum1 == lum2:
13                    count = count+1
14                if count == k:
15                    p=p+1
16        return p
```

K = 4

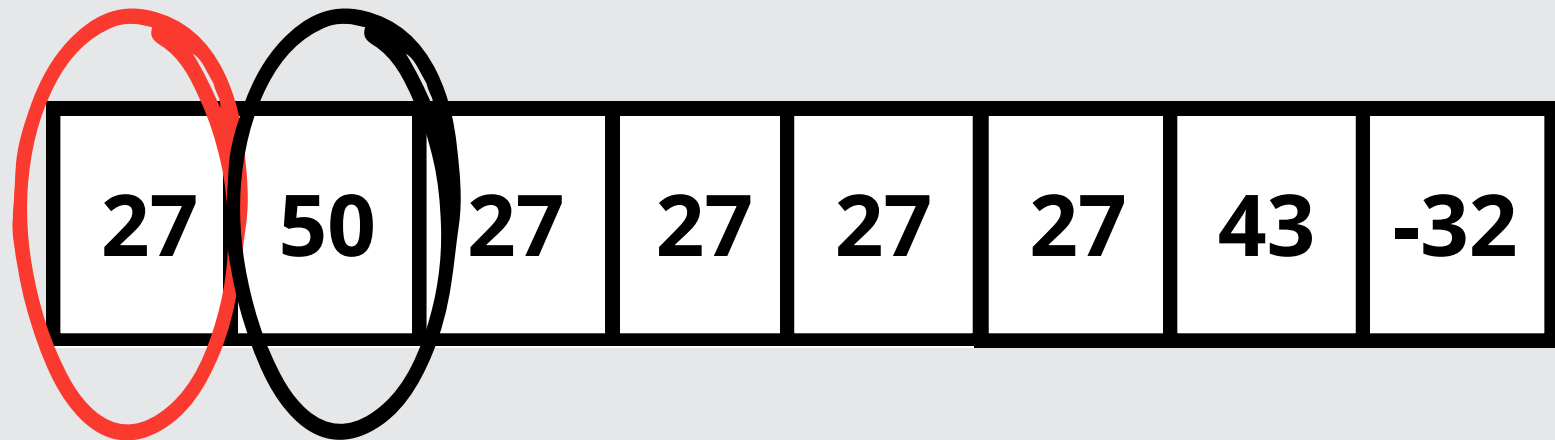


- Count = 0
- p = 0

Caccia all'errore

```
1 def contaPix(L, k):
2     #@ param L : sequence of pixel
3     #@ param k: int
4     #@ return int
5     p = 0
6     for i in range(0, len(L)):
7         count = 0
8         lum1 = lum(L[i])
9         for j in range(0, len(L)):
10            if i != j:
11                lum2 = lum(L[j])
12                if lum1 == lum2:
13                    count = count+1
14                    if count == k:
15                        p=p+1
16            return p
```

K = 4

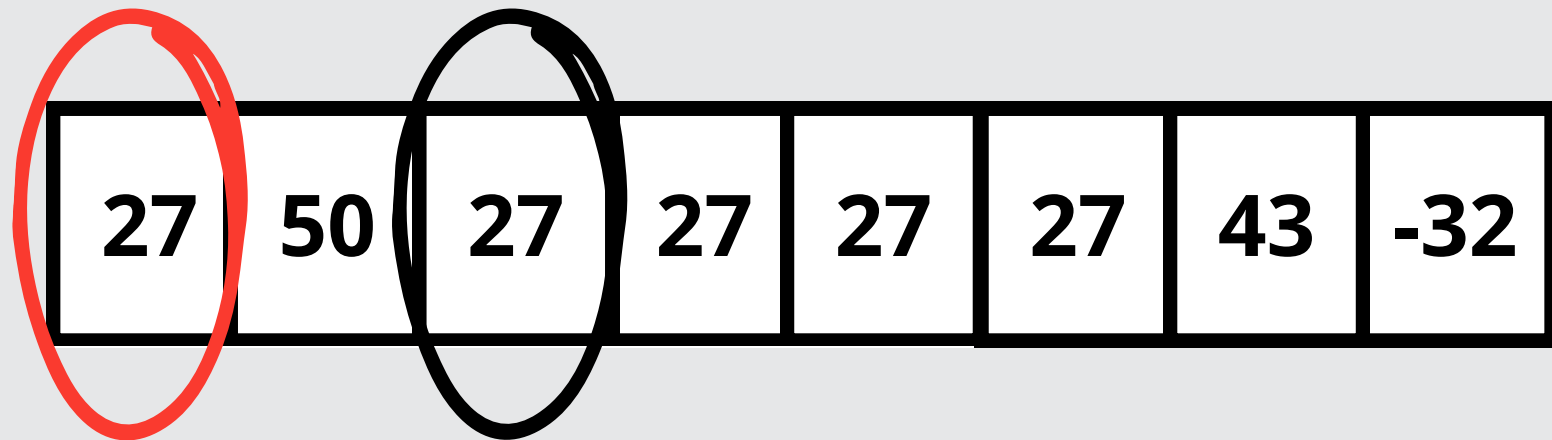


- **Count = 0**
- **p = 0**

Caccia all'errore

```
1 def contaPix(L, k):
2     #@ param L : sequence of pixel
3     #@ param k: int
4     #@ return int
5     p = 0
6     for i in range(0, len(L)):
7         count = 0
8         lum1 = lum(L[i])
9         for j in range(0, len(L)):
10            if i != j:
11                lum2 = lum(L[j])
12                if lum1 == lum2:
13                    count = count+1
14                if count == k:
15                    p=p+1
16        return p
```

K = 4

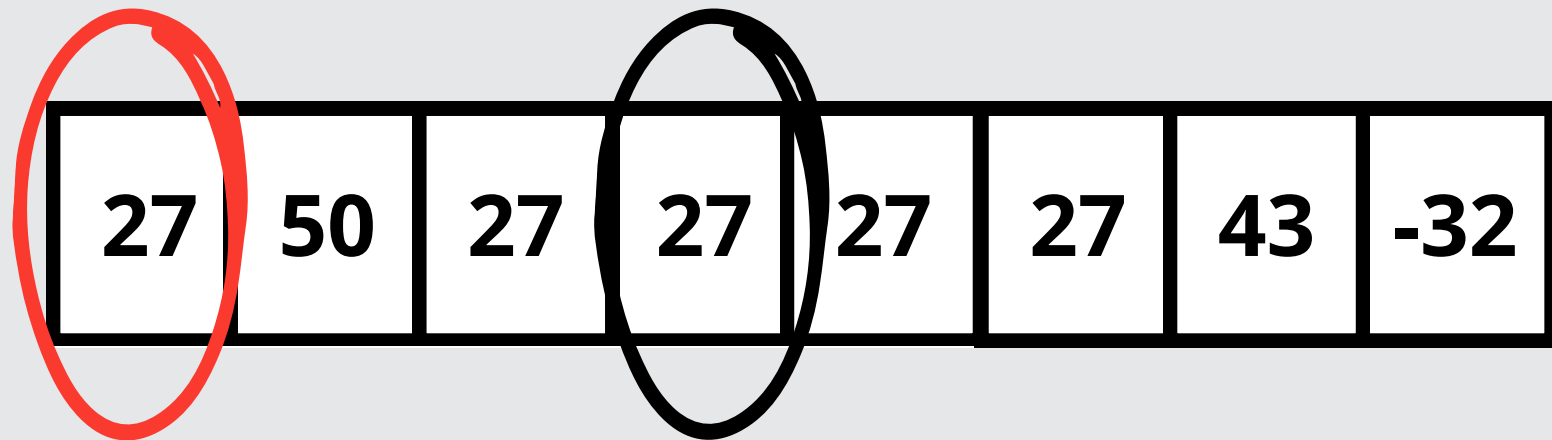


- **Count = 1**
- **p = 0**

Caccia all'errore

```
1 def contaPix(L, k):
2     #@ param L : sequence of pixel
3     #@ param k: int
4     #@ return int
5     p = 0
6     for i in range(0, len(L)):
7         count = 0
8         lum1 = lum(L[i])
9         for j in range(0, len(L)):
10            if i != j:
11                lum2 = lum(L[j])
12                if lum1 == lum2:
13                    count = count+1
14                    if count == k:
15                        p=p+1
16            return p
```

K = 4

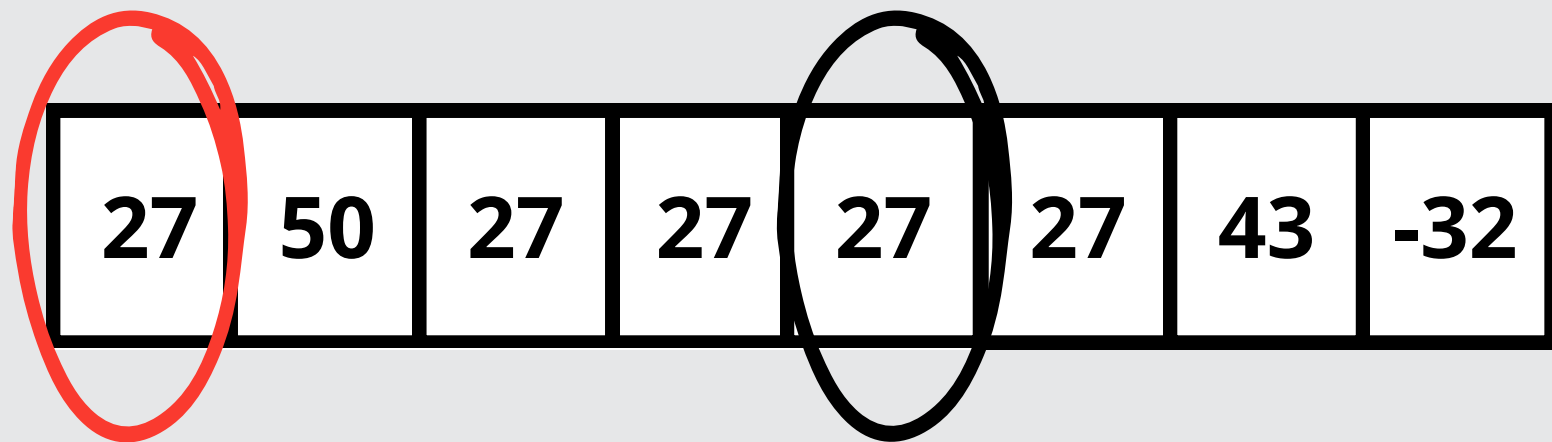


- **Count = 2**
- **p = 0**

Caccia all'errore

```
1 def contaPix(L, k):
2     #@ param L : sequence of pixel
3     #@ param k: int
4     #@ return int
5     p = 0
6     for i in range(0, len(L)):
7         count = 0
8         lum1 = lum(L[i])
9         for j in range(0, len(L)):
10            if i != j:
11                lum2 = lum(L[j])
12                if lum1 == lum2:
13                    count = count+1
14                if count == k:
15                    p=p+1
16        return p
```

K = 4

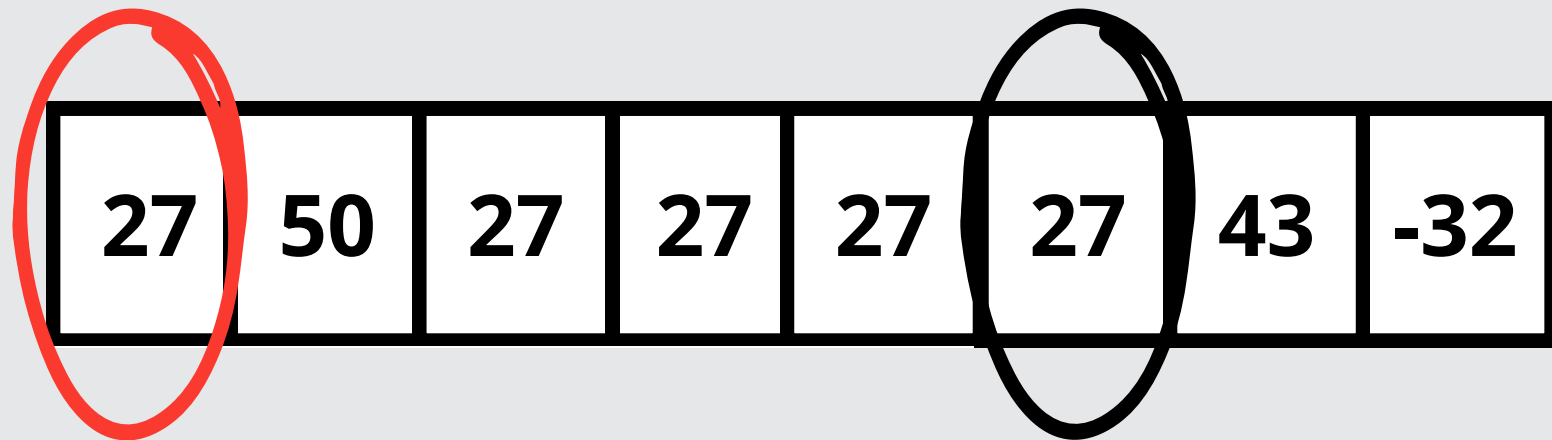


- Count =3
- p = 0

Caccia all'errore

```
1 def contaPix(L, k):
2     #@ param L : sequence of pixel
3     #@ param k: int
4     #@ return int
5     p = 0
6     for i in range(0, len(L)):
7         count = 0
8         lum1 = lum(L[i])
9         for j in range(0, len(L)):
10            if i != j:
11                lum2 = lum(L[j])
12                if lum1 == lum2:
13                    count = count+1
14                    if count == k:
15                        p=p+1
16            return p
```

K = 4

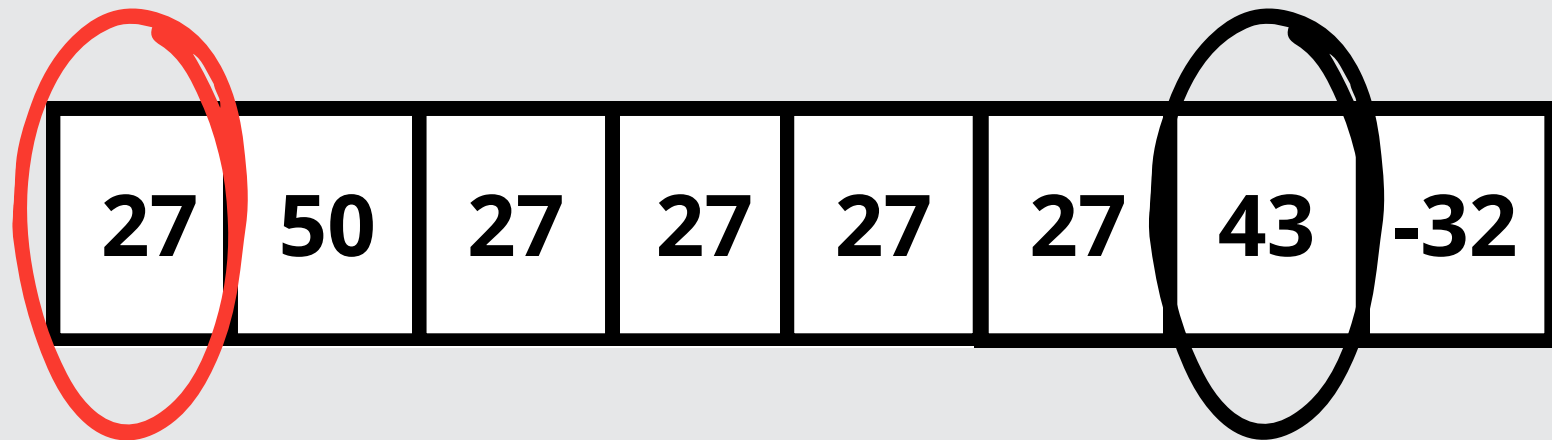


- **Count = 4**
- **p = 1**

Caccia all'errore

```
1 def contaPix(L, k):
2     #@ param L : sequence of pixel
3     #@ param k: int
4     #@ return int
5     p = 0
6     for i in range(0, len(L)):
7         count = 0
8         lum1 = lum(L[i])
9         for j in range(0, len(L)):
10            if i != j:
11                lum2 = lum(L[j])
12                if lum1 == lum2:
13                    count = count+1
14                if count == k:
15                    p=p+1
16        return p
```

K = 4

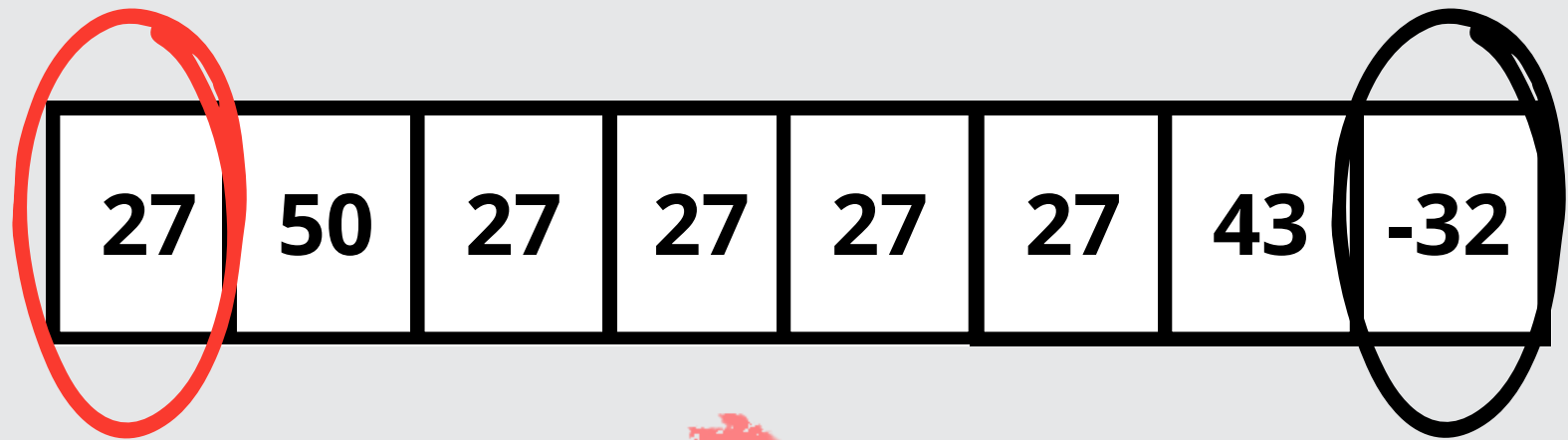


- **Count =4**
- **p = 2**

Caccia all'errore

```
1 def contaPix(L, k):
2     #@ param L : sequence of pixel
3     #@ param k: int
4     #@ return int
5     p = 0
6     for i in range(0, len(L)):
7         count = 0
8         lum1 = lum(L[i])
9         for j in range(0, len(L)):
10            if i != j:
11                lum2 = lum(L[j])
12                if lum1 == lum2:
13                    count = count+1
14                    if count == k:
15                        p=p+1
16            return p
```

K = 4



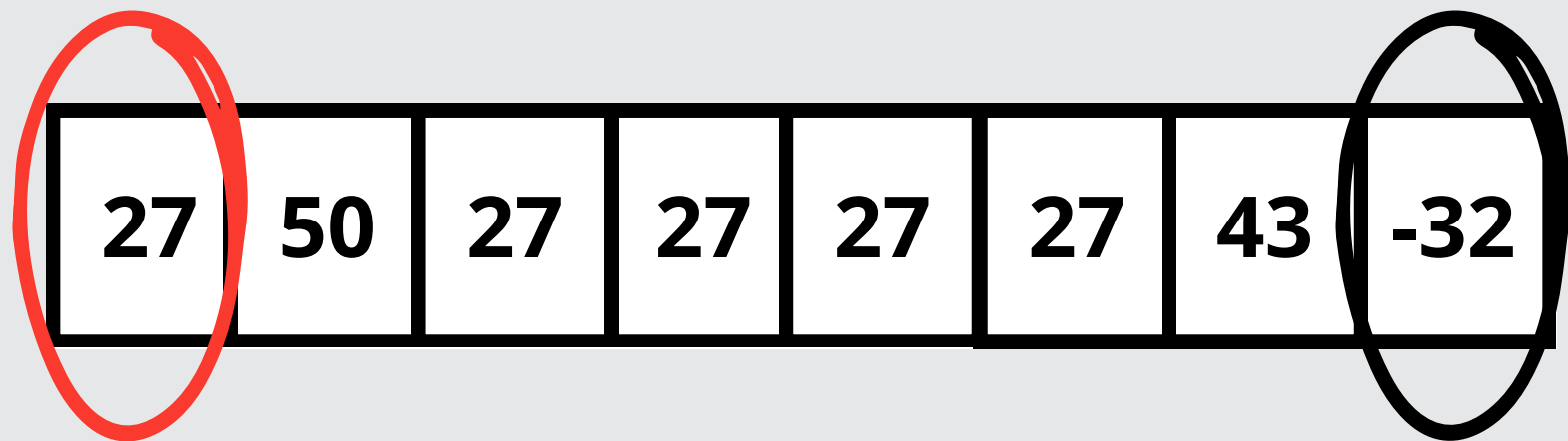
- Count = 4
- p = 3



Caccia all'errore

```
1 def contaPix(L, k):
2     #@ param L : sequence of pixel
3     #@ param k: int
4     #@ return int
5     p = 0
6     for i in range(0, len(L)):
7         count = 0
8         lum1 = lum(L[i])
9         for j in range(0, len(L)):
10            if i != j:
11                lum2 = lum(L[j])
12                if lum1 == lum2:
13                    count = count+1
14                ← if count == k:
15                    p=p+1
16            return p
```

K = 4



- Count = 4
- p = 3

Deve essere fatto fuori dal ciclo!

Caccia all'errore: soluzione

```
1  def contaPix(L, k):
2  #@ param L : sequence of pixel
3  #@ param k: int
4  ▼ #@ return int
5      p = 0
6  ▼ for i in range(0, len(L)):
7      count = 0
8      lum1 = lum(L[i])
9  ▼ for j in range (0, len(L)):
10 ▼     if i != j:
11         lum2 = lum(L[j])
12 ▼         if lum1 == lum2:
13             count = count+1
14 ▼         if count == k:
15             p=p+1
16     return p
17
18
19  def lum(p):
20  #@ param p : pixel
21 ▼ #@ return int
22     return (getRed(p)+getGreen(p)+getBlue(p))
```

Riconoscimento contorni:

```
def lineDetect(..., threshold):  
    ...  
    # @param threshold: ...  
    # @return ...  
    makeBW = duplicatePicture(orig) #makeBW e' l'immagine che verra' modificata tracciando i bordi  
    for ... in range(0, getWidth(orig)-1):  
        for y in range(0, ... -1):  
            here=getPixel(makeBW, x, y)  
            down=getPixel(orig, x, y+1)  
            right=getPixel(..., x+1, y)  
            hereLum=(getRed(here)+getGreen(here)+getBlue(here))/3  
            downLum=(getRed(down)+getGreen(...)+getBlue(...))/3  
            rightLum=(getRed(...)+getGreen(...)+getBlue(...))/3  
            if (abs(hereLum-downLum)>threshold) and (abs(hereLum-rightLum)>threshold):  
                setColor(..., black)  
            if (abs(hereLum-downLum)<=threshold) or (abs(hereLum-rightLum)<=threshold):  
                setColor(..., white)  
    return makeBW
```


Riconoscimento contorni:

```
def lineDetect(, threshold):
    # @param threshold:
    # @return
    makeBW = duplicatePicture(orig) #makeBW e' l'immagine che verra' modificata tracciando i bordi
    for x in range(0, getWidth(orig)-1):
        for y in range(0, -1):
            here=getPixel(makeBW, x, y)
            down=getPixel(orig, x, y+1)
            right=getPixel(, x+1, y)
            hereLum=(getRed(here)+getGreen(here)+getBlue(here))/3
            downLum=(getRed(down)+getGreen()+getBlue())/3
            rightLum=(getRed()+getGreen()+getBlue())/3
            if (abs(hereLum-downLum)>threshold) and (abs(hereLum-rightLum)>threshold):
                setColor(, black)
            if (abs(hereLum-downLum)<=threshold) or (abs(hereLum-rightLum)<=threshold):
                setColor(, white)
    return makeBW
```

Riconoscimento contorni:

```
def lineDetect(orig, threshold):  
    # @param threshold:   
    # @return   
    makeBW = duplicatePicture(orig) #makeBW e' l'immagine che verra' modificata tracciando i bordi  
    for x in range(0, getWidth(orig)-1):  
        for y in range(0,   
            here=getPixel(makeBW, x, y)  
            down=getPixel(orig, x, y+1)  
            right=getPixel(  
            hereLum=(getRed(here)+getGreen(here)+getBlue(here))/3  
            downLum=(getRed(down)+getGreen(  
            rightLum=(getRed(  
            if (abs(hereLum-downLum)>threshold) and (abs(hereLum-rightLum)>threshold):  
                setColor(  
            if (abs(hereLum-downLum)<=threshold) or (abs(hereLum-rightLum)<=threshold):  
                setColor(  
    return makeBW
```

Riconoscimento contorni:

```
def lineDetect(orig, threshold):
# @param orig:Picture
# @param threshold: float
# @return Picture
makeBW = duplicatePicture(orig) #makeBW e' l'immagine che verra' modificata tracciando i bordi
for x in range(0, getWidth(orig)-1):
    for y in range(0, [REDACTED]-1):
        here=getPixel(makeBW, x, y)
        down=getPixel(orig, x, y+1)
        right=getPixel([REDACTED], x+1, y)
        hereLum=(getRed(here)+getGreen(here)+getBlue(here))/3
        downLum=(getRed(down)+getGreen([REDACTED])+getBlue([REDACTED]))/3
        rightLum=(getRed([REDACTED])+getGreen([REDACTED])+getBlue([REDACTED]))/3
        if (abs(hereLum-downLum)>threshold) and (abs(hereLum-rightLum)>threshold):
            setColor([REDACTED], black)
        if (abs(hereLum-downLum)<=threshold) or (abs(hereLum-rightLum)<=threshold):
            setColor([REDACTED], white)
    return makeBW
```

Riconoscimento contorni:

```
def lineDetect(orig, threshold):
# @param orig:Picture
# @param threshold: float
# @return Picture
makeBW = duplicatePicture(orig) #makeBW e' l'immagine che verra' modificata tracciando i bordi
for x in range(0, getWidth(orig)-1):
    for y in range(0, getHeight(orig)-1):
        here=getPixel(makeBW, x, y)
        down=getPixel(orig, x, y+1)
        right=getPixel(      x+1, y)
        hereLum=(getRed(here)+getGreen(here)+getBlue(here))/3
        downLum=(getRed(down)+getGreen(      )+getBlue(      ))/3
        rightLum=(getRed(      )+getGreen(      )+getBlue(      ))/3
        if (abs(hereLum-downLum)>threshold) and (abs(hereLum-rightLum)>threshold):
            setColor(      , black)
        if (abs(hereLum-downLum)<=threshold) or (abs(hereLum-rightLum)<=threshold):
            setColor(      , white)
    return makeBW
```

Riconoscimento contorni:

```
def lineDetect(orig, threshold):
# @param orig:Picture
# @param threshold: float
# @return Picture
makeBW = duplicatePicture(orig) #makeBW e' l'immagine che verra' modificata tracciando i bordi
for x in range(0, getWidth(orig)-1):
    for y in range(0, getHeight(orig)-1):
        here=getPixel(makeBW, x, y)
        down=getPixel(orig, x, y+1)
        right=getPixel(orig, x+1, y)
        hereLum=(getRed(here)+getGreen(here)+getBlue(here))/3
        downLum=(getRed(down)+getGreen( )+getBlue( ))/3
        rightLum=(getRed( )+getGreen( )+getBlue( ))/3
        if (abs(hereLum-downLum)>threshold) and (abs(hereLum-rightLum)>threshold):
            setColor( , black)
        if (abs(hereLum-downLum)<=threshold) or (abs(hereLum-rightLum)<=threshold):
            setColor( , white)
    return makeBW
```


Riconoscimento contorni:

```
def lineDetect(orig, threshold):
# @param orig:Picture
# @param threshold: float
# @return Picture
makeBW = duplicatePicture(orig) #makeBW e' l'immagine che verra' modificata tracciando i bordi
for x in range(0, getWidth(orig)-1):
    for y in range(0, getHeight(orig)-1):
        here=getPixel(makeBW, x, y)
        down=getPixel(orig, x, y+1)
        right=getPixel(orig, x+1, y)
        hereLum=(getRed(here)+getGreen(here)+getBlue(here))/3
        downLum=(getRed(down)+getGreen(down)+getBlue(down))/3
        rightLum=(getRed( )+getGreen( )+getBlue( ))/3
        if (abs(hereLum-downLum)>threshold) and (abs(hereLum-rightLum)>threshold):
            setColor( , black)
        if (abs(hereLum-downLum)<=threshold) or (abs(hereLum-rightLum)<=threshold):
            setColor( , white)
    return makeBW
```


Riconoscimento contorni:

```
def lineDetect(orig, threshold):
# @param orig:Picture
# @param threshold: float
# @return Picture
makeBW = duplicatePicture(orig) #makeBW e' l'immagine che verra' modificata tracciando i bordi
for x in range(0, getWidth(orig)-1):
    for y in range(0, getHeight(orig)-1):
        here=getPixel(makeBW, x, y)
        down=getPixel(orig, x, y+1)
        right=getPixel(orig, x+1, y)
        hereLum=(getRed(here)+getGreen(here)+getBlue(here))/3
        downLum=(getRed(down)+getGreen(down)+getBlue(down))/3
        rightLum=(getRed(right)+getGreen(right)+getBlue(right))/3
        if (abs(hereLum-downLum)>threshold) and (abs(hereLum-rightLum)>threshold):
            setColor(■, black)
        if (abs(hereLum-downLum)<=threshold) or (abs(hereLum-rightLum)<=threshold):
            setColor(■, white)
    return makeBW
```

Riconoscimento contorni: soluzione

```
def lineDetect(orig, threshold):
    # @param orig:Picture
    # @param threshold: float
    # @return Picture
    makeBW = duplicatePicture(orig) #makeBW e' l'immagine che verra' modificata tracciando i bordi
    for x in range(0, getWidth(orig)-1):
        for y in range(0, getHeight(orig)-1):
            here=getPixel(makeBW, x, y)
            down=getPixel(orig, x, y+1)
            right=getPixel(orig, x+1, y)
            hereLum=(getRed(here)+getGreen(here)+getBlue(here))/3
            downLum=(getRed(down)+getGreen(down)+getBlue(down))/3
            rightLum=(getRed(right)+getGreen(right)+getBlue(right))/3
            if (abs(hereLum-downLum)>threshold) and (abs(hereLum-rightLum)>threshold):
                setColor(here, black)
            if (abs(hereLum-downLum)<=threshold) or (abs(hereLum-rightLum)<=threshold):
                setColor(here, white)
    return makeBW
```


Esercizio aggiuntivo:

Creare una funzione che, data un'immagine, restituisca l'occorrenza dei pixel con la luminosità minore.

N.B. la luminosità di un pixel è calcolata come la somma delle tre componenti del colore del pixel.

Luminosità = componente rossa + componente blu +
componente verde